

Learnlytics

handbook

Mathematics & Statistics For DS



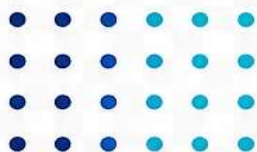
Concise



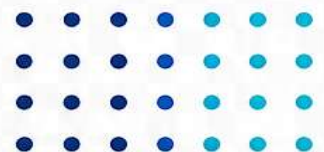
Comprehensive



Career-Defining



LEARN SMART.
ANALYZE BETTER.
GROW FASTER.



MATHEMATICS FOR DATA SCIENCE

PART 02

A Complete 13-Chapter Solution Manual

First Edition

Published by Learnlytics.handbook

www.learnlyticshandbook.shop | learnlytic.handbook@gmail.com

Copyright © 2026 Learnlytics.handbook. All rights reserved.

No part of this publication may be reproduced, distributed, stored in a retrieval system, or transmitted in any form or by any means — electronic, mechanical, photocopying, recording, or otherwise — without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

For permission requests or licensing inquiries, write to the publisher at the address above.

ISBN: 978-1-2565-5895-9 (Digital Edition)

First published: 2026

Printed and distributed digitally worldwide.

IMPORTANT: LICENSE & COPYRIGHT NOTICE This document and all included materials are the exclusive intellectual property of Learnlytics.handbook. Your purchase grants you a single-user, personal license. This file is digitally tracked. Unauthorized reproduction, forwarding, resale, or distribution of this material — whether in private groups, Telegram channels, or public forums — is strictly prohibited and constitutes copyright infringement. Our legal team actively monitors digital distribution networks. Violations of this agreement will result in immediate termination of your access, notification to your educational/professional institution, and formal legal action under applicable international and domestic copyright laws. Thank you for respecting the hard work that went into creating this guide!

Disclaimer: The information in this guide is provided for educational purposes. Mathematical concepts and statistical methods presented here reflect widely accepted academic standards. Always refer to your institution's or organisation's preferred references for domain-specific notation. The author and publisher make no warranty, express or implied, with respect to the material contained herein.

Typeset in Arial | Cover design: Learnlytics.handbook

IN Made in India with  for Data Science Worldwide

TABLE OF CONTENTS

13 Chapters • 260 QS • A Complete Math for ML Journey

PART 2: Mathematics for Machine Learning

	CHAPTER TITLE	PAGE
1	Chapter 1: Vectors, Matrices & Operations	04-17
2	Chapter 2: Matrix Inverse & Determinant	18-33
3	Chapter 3: Eigenvalues & Eigenvectors	34-49
4	Chapter 4: SVD — Singular Value Decomposition	50-66
5	Chapter 5: Projections & Orthogonality	67-85
6	Chapter 6: Cosine Similarity	86-107
7	Chapter 7: Gradient Vectors & Hessian	108-128
8	Chapter 8: Derivatives & Chain Rule	129-151
9	Chapter 9: Partial Derivatives & Gradient	152-175
10	Chapter 10: Gradient Descent & Variants	176-198
11	Chapter 11: Convexity & Loss Functions	199-220
12	Chapter 12: Backpropagation Intuition	221-243
13	Chapter 13: Numerical Optimisation Methods	244-267

Chapter 1 Test Questions

Vectors, Matrices & Operations

Warmup Test + Interviewer Test — 20 Questions Total

How These Answers Are Written

For every Descriptive, Coding, and Scenario-Based question, the answer is written the way a thoughtful student would actually explain it in an interview — conversational, walking through the reasoning, admitting where mistakes commonly happen, and backing it up with a short, focused code snippet. The goal is not to recite the textbook definition but to demonstrate genuine understanding of the 'why' behind each concept. MCQ questions are answered directly with the correct option marked and a brief one-line rationale.

Level 1: Warmup Test (10 Questions)

Covers the foundational definitions and numpy mechanics from Sections 1.1–1.8: vectors, matrices, core operations, matrix multiplication, transpose, and broadcasting.

Section A — Multiple Choice Questions (5)

Q1. What is the shape of the numpy array created by `np.array([1, 2, 3])`?

- A) (1, 3)
- B) (3, 1)
- C) (3,) ✓ Correct**
- D) (1, 3, 1)

Why: A 1D numpy array has no row/column distinction — shape (3,) is NOT the same as (1,3) or (3,1). This is the most common source of silent shape bugs for beginners.

Q2. Which operator performs TRUE matrix multiplication in numpy (not element-wise)?

- A) *
- B) @ ✓ Correct**
- C) `np.add()`
- D) `np.multiply()`

Why: @ (or `np.dot()`) follows the $(m \times k) @ (k \times n) \rightarrow (m \times n)$ rule. * does Hadamard (element-wise) multiplication and requires identical shapes.

Q3. If A has shape (4, 6) and B has shape (6, 3), what is the shape of `A @ B`?

- A) (4, 3) ✓ Correct**
- B) (6, 6)
- C) (3, 4)
- D) Error — shapes don't align

Why: Inner dimensions (6 and 6) match and cancel out; the outer dimensions (4 and 3) form the result shape.

Q4. What is the correct transpose rule for a matrix product $(AB)^T$?

- A) $(AB)^T = A^T B^T$
B) $(AB)^T = B^T A^T$ ✓ Correct
 C) $(AB)^T = (AB)^{-1}$
 D) $(AB)^T = BA$

Why: Order reverses when transposing a product — like taking off shoes before socks. This shows up constantly in backpropagation derivations.

Q5. What happens when you compute $(1000, 5) - (5,)$ in numpy?

- A) ValueError — shapes incompatible
B) numpy broadcasts the (5,) array across all 1000 rows, result is (1000,5) ✓ Correct
 C) Only the first 5 rows are affected
 D) numpy reshapes to (1000,1000)

Why: This is exactly how feature standardization works — broadcasting stretches the smaller array along the missing/size-1 dimension to match the larger one.

Section B — Descriptive Questions (3)

Q6. [Descriptive]

Explain the difference between element-wise multiplication (Hadamard product) and matrix multiplication in numpy. Why does mixing them up cause silent bugs?

Answer

So this is honestly one of the first things that tripped me up when I started with numpy, and it's worth slowing down on. $A * B$ does element-wise multiplication — it just multiplies whatever's in position $[i,j]$ of A by whatever's in position $[i,j]$ of B. For that to make sense, A and B need the exact same shape. $A @ B$, on the other hand, is real matrix multiplication — $C[i,j]$ becomes the dot product of row i of A with column j of B, following the $(m \times k) @ (k \times n) \rightarrow (m \times n)$ rule.

Why does mixing them up bite people? Because numpy won't complain if both matrices happen to be square and the same shape — $A * B$ and $A @ B$ will both run without error, just give completely different (and wrong) numbers. I've seen this happen in someone's custom gradient computation where they used $*$ instead of $@$ for a weight update that needed a real matrix product — the model trained, loss went down a bit, but the results were just off. No crash, no warning, just silently wrong math. That's the scary part — shape errors are loud, but the $*$ vs $@$ mistake is quiet.

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

A * B # [[5, 12], [21, 32]] -- element-wise
A @ B # [[19, 22], [43, 50]] -- matrix multiplication
```

Q7. [Descriptive]

Why is the dot product described as 'the single most important operation in ML'? Give two concrete examples from real ML workflows.

Answer

When I first read that line I thought it was a bit of an overstatement, but the more I've worked with this stuff the more I think it's true. The dot product does one simple thing — measure how much two vectors 'agree' — but that one operation gets reused everywhere because so much of ML is fundamentally about comparing things or combining weighted signals.

Two examples I keep running into: first, linear regression prediction is literally just a dot product — $\hat{y} = w \cdot x$. You're taking your learned weights and asking 'how much does this row of features align with what I've learned matters?' Second, attention in transformers — the famous QK^T — is computing a dot product between a query vector and a key vector for every pair of tokens, asking 'how relevant is token A to token B?' Once you see that pattern, you start spotting dot products everywhere: cosine similarity, neural network layers, kernel methods, even gradient computations.

```
import numpy as np
w = np.array([0.5, -0.3, 1.2])
x = np.array([2.0, 1.0, 0.5])
y_pred = np.dot(w, x) # = w . x -- the entire 'model' in one line
```

Q8. [Descriptive]

Explain what feature standardization (Z-score scaling) does mathematically, and why distance-based algorithms like KNN break down without it.

Answer

The formula is simple — $(X - \text{mean}) / \text{std}$ — but the intuition matters more than the formula itself. You're shifting every feature so its mean becomes 0, then stretching or squashing it so its standard deviation becomes 1. After that, a value of +2 means the same thing for every feature: 'two standard deviations above average,' regardless of whether the original feature was age (0-100) or income (0-200,000).

Why does this matter for KNN specifically? KNN finds the 'nearest' points using Euclidean distance, and that formula sums up squared differences across features. If one feature is on a scale of thousands (salary) and another is on a scale of tens (age), the squared salary differences will be enormous compared to age differences — so salary basically becomes the only feature that matters, even if age is actually more predictive. Standardizing puts every feature on equal footing before any distance is computed.

```
import numpy as np
X = np.random.randn(100, 5)
mean = X.mean(axis=0) # shape (5,)
std = X.std(axis=0) # shape (5,)
X_scaled = (X - mean) / std # broadcasting handles the shape mismatch
```

Section C — Coding Questions (2)

Q9. [Coding]

Write numpy code to compute the cosine similarity between vectors $a=[1,2,3]$ and $b=[4,5,6]$ WITHOUT using sklearn. Briefly explain why you can't just use `np.dot()` alone.

Answer

The key thing to remember is that the raw dot product on its own isn't similarity — it's contaminated by magnitude. Two vectors pointing in the exact same direction but with very different lengths will give a big dot product even though they're 'the same' direction-wise, and two short vectors pointing the same way will give a small dot product even though they're just as aligned. So you have to divide out both magnitudes to isolate just the angle between them.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

cos_sim = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print(cos_sim) # 0.9746 -- close to 1, vectors point in a similar direction
```

One thing I always double-check: if either vector could be all zeros, you'll get a divide-by-zero, so in production code I'd add a small epsilon to the denominator or check for it explicitly.

Q10. [Coding]

Given a feature matrix X of shape $(100, 5)$, write code to manually standardize it (Z-score) and explain what would go wrong if you used `X.mean()` instead of `X.mean(axis=0)`.

Answer

This one's about being careful with the axis argument — a small thing that completely changes what you're computing. `X.mean(axis=0)` collapses down the rows and gives you one mean PER COLUMN (per feature) — a $(5,)$ array, exactly what you want for per-feature standardization. If you just do `X.mean()` with no axis, numpy collapses the WHOLE matrix into a single number — the overall average across every value, ignoring which feature it came from. That standardizes everything against one global mean and std, defeating the purpose, since different features are on totally different scales to begin with.

```
import numpy as np
```

```
X = np.random.randn(100, 5)

mean = X.mean(axis=0) # shape (5,) -- one mean per feature, correct
std = X.std(axis=0) # shape (5,) -- one std per feature, correct

X_scaled = (X - mean) / std # (100,5) - (5,) broadcasts to (100,5)

# WRONG version:
wrong_mean = X.mean() # single scalar -- collapses everything, loses per-
feature meaning
```

Level 2: Interviewer Test (10 Intermediate / Semi-Advanced Questions)

Applies Chapter 1 concepts to realistic debugging scenarios, model-building tasks, and database-style data — the kind of questions that come up in actual interviews and on-the-job debugging.

Section A — Scenario-Based Multiple Choice Questions (3)

Q1. Scenario: You have a dataset X with shape $(200, 10)$ — 200 samples, 10 features — and a weight vector w with shape $(10,)$. You want predictions for all samples. Which line of code is correct?

A) `y_pred = w @ X`

B) `y_pred = X @ w` ✓ Correct

C) `y_pred = X * w`

D) `y_pred = w.T @ X.T`

Why: $(200, 10) @ (10,) \rightarrow (200,)$. Option A would attempt $(10,) @ (200, 10)$, which fails the inner-dimension rule. Option C is element-wise and doesn't give per-sample weighted sums.

Q2. Scenario: A teammate computes a covariance matrix using $X @ X$ instead of $X.T @ X$, where X is shape $(50, 50)$ — 50 samples, 50 features, same number by coincidence. What actually happens?

A) Raises a `ValueError` because the shapes can never multiply

B) Silently runs and returns a $(50, 50)$ matrix, but it's NOT a real feature-covariance matrix — it's a completely different quantity (sample-to-sample, not feature-to-feature) ✓ Correct

C) Returns the exact same answer as $X.T @ X$ for any matrix

D) numpy detects this is a covariance computation and auto-corrects to $X.T @ X$

Why: Because n happens to equal p ($50=50$), the shapes 'match' by coincidence, so numpy runs without complaint — but the result is mathematically NOT covariance. This is the dangerous case: no crash, just silently wrong math. If $n \neq p$ you'd get a loud (and safer) `ValueError` instead.

Q3. Scenario: Your KNN model's predictions are almost entirely driven by the 'salary' feature (30,000–200,000) and barely affected by 'age' (18–80) or 'years_experience' (0–40). What's the root cause and best fix?

A) KNN is fundamentally a bad algorithm for this data — switch models

B) The Euclidean distance is dominated by the largest-magnitude feature (salary); fix by standardizing all features before computing distances ✓ Correct

C) Increase k (number of neighbours) to balance the effect

D) Remove the salary feature entirely since it's causing problems

Why: Euclidean distance sums squared differences, so a feature on a scale of thousands produces squared differences orders of magnitude larger than a feature on a scale of tens. Standardizing puts every feature on comparable footing before distances are computed.

Section B — Scenario-Based Descriptive Questions (2)

Q4. [Scenario — Descriptive]

You're building a 2-layer neural network forward pass manually in numpy. X has shape (64, 20) and you write $h = X @ W1 + b1$ where W1 is (20, 32) and b1 is (32,) — this works fine. Then for the second layer you write $y_pred = h @ W2 + b2$ and get a 'shapes not aligned' error. Walk through your debugging process and explain the fix.

Answer

The very first thing I'd do — before even reading the error message in detail — is print every shape involved. That's rule one whenever numpy throws a shape error.

```
print(h.shape) # (64, 32) -- output of layer 1
print(W2.shape) # let's say (16, 1) -- this is the bug
```

h is (64, 32) because layer 1's hidden size was 32 (W1's second dimension). For $h @ W2$ to work, W2's FIRST dimension has to match h's SECOND dimension — meaning W2 needs to start with 32, not whatever it currently is. If someone wrote $W2 = np.random.randn(16, 1)$ by mistake — maybe copy-pasted from a different architecture, or just miscounted hidden units — that's exactly the inner-dimension mismatch that throws this error.

```
W2 = np.random.randn(32, 1) # now matches h's second dimension (32)
b2 = np.zeros(1)
y_pred = h @ W2 + b2 # (64,32) @ (32,1) -> (64,1) -- correct!
```

This is honestly the #1 bug I run into building networks by hand — the output size of one layer HAS to equal the input size of the next layer, and it's easy to mistype when stacking several layers quickly. My habit now is to literally write the shape as a comment next to every weight matrix definition, so a mismatch jumps out visually before I even run the code.

Q5. [Scenario — Descriptive]

A junior teammate computed similarity between two product description vectors using just $np.dot(a, b)$ and got a similarity score of 850 for one pair and 12 for another, concluding the first pair is 'way more similar.' You suspect this conclusion might be wrong. Explain what's likely happening and how you'd correct it.

Answer

My first instinct is that this is almost certainly a magnitude problem, not a similarity problem. The raw dot product doesn't measure 'how similar in direction' two vectors are — it measures direction AND magnitude mixed together. If vector a (maybe a longer, more verbose product description, leading to bigger TF-IDF counts) just happens to have larger numbers overall, the dot product will be big even if the actual topical overlap with b is mediocre. Meanwhile a shorter description compared

to something else could have a small dot product even if they're describing nearly identical products, simply because the raw counts are smaller.

The fix is to normalize — divide the dot product by the product of both vectors' magnitudes. That's literally the definition of cosine similarity, and it isolates the ANGLE between vectors from their LENGTH.

```
import numpy as np
a = np.array([120, 95, 60]) # raw counts -- could be big or small
b = np.array([8, 6, 5]) # much smaller raw counts

raw_dot = np.dot(a, b) # influenced by magnitude
cos_sim = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)) # isolates
direction only
print(raw_dot, cos_sim)
```

I'd walk my teammate through recomputing both pairs with cosine similarity instead of the raw dot product, and I'd bet the 'way more similar' pair from before turns out much closer to the other pair once magnitude is taken out of the picture — or even flips entirely.

Section C — Coding Questions (3)

Q6. [Coding]

Implement linear regression's closed-form normal equation, including the bias trick (adding a column of ones to X so you don't need a separate intercept term), and use it to predict on new data.

Answer

The 'bias trick' is one of those small things that makes a big difference — without it, the model is forced to fit a hyperplane through the origin, which almost never matches real data. Sticking a column of all 1s onto X gives the model an extra 'always-on' feature, and the weight learned for that column becomes the intercept.

```
import numpy as np

def fit_linear_regression(X, y):
    n = X.shape[0]
    X_aug = np.hstack([X, np.ones((n, 1))]) # bias trick: add column of 1s
    # w* = (X^T X)^-1 X^T y -- but lstsq is more numerically stable than inv()
    w_aug, _, _, _ = np.linalg.lstsq(X_aug, y, rcond=None)
    w, b = w_aug[:-1], w_aug[-1] # split weights and intercept
    return w, b

def predict(X_new, w, b):
```

```

return X_new @ w + b

# Example
X = np.random.randn(100, 3)
y = X @ np.array([2.0, -1.0, 0.5]) + 3.0 + 0.1*np.random.randn(100)

w, b = fit_linear_regression(X, y)
print('weights:', w.round(3), ' intercept:', round(b, 3))

X_new = np.random.randn(5, 3)
preds = predict(X_new, w, b)
print('predictions:', preds.round(3))

```

I used `lstsq` instead of literally inverting $X.T @ X$ — that inverse can blow up numerically if features are correlated, and `lstsq` handles that more gracefully under the hood via SVD.

Q7. [Coding]

Write a function that computes the full pairwise cosine similarity matrix for all rows of a matrix X — no Python for-loops, just broadcasting and matrix multiplication.

Answer

The trick is to first normalize every row to unit length, then just do a single matrix multiplication against its own transpose. Once every row has length 1, the dot product between any two rows IS the cosine similarity directly — no need to divide by norms inside a loop.

```

import numpy as np

def cosine_similarity_matrix(X):
    norms = np.linalg.norm(X, axis=1, keepdims=True) # shape (n,1) -- per-row norm
    X_norm = X / norms # broadcasts to (n,d), unit-length rows
    return X_norm @ X_norm.T # (n,d) @ (d,n) -> (n,n) all-pairs

X = np.random.randn(6, 4)
sim_matrix = cosine_similarity_matrix(X)
print(sim_matrix.shape) # (6, 6)
print(np.diag(sim_matrix).round(4)) # should be all 1.0 -- every vector vs itself

```

Two sanity checks I always run: the diagonal should be exactly 1.0 (every vector is perfectly similar to itself), and the matrix should be symmetric (`sim_matrix[i,j] == sim_matrix[j,i]`) since similarity doesn't care about order.

Q8. [Coding]

Implement a 2-layer neural network forward pass manually (no PyTorch/TensorFlow), given input X and weight/bias parameters for two layers, using ReLU activation on the hidden layer.

Answer

This is really just two matrix multiplications with a non-linearity squeezed between them — that's the entire 'magic' of a basic feedforward network when you strip away the training part.

```
import numpy as np

def relu(z):
    return np.maximum(0, z)

def forward_pass(X, W1, b1, W2, b2):
    z1 = X @ W1 + b1 # (batch,in)@(in,hidden)+(hidden,) -> (batch,hidden)
    h1 = relu(z1) # non-linearity -- without this, layers collapse into one
    z2 = h1 @ W2 + b2 # (batch,hidden)@(hidden,out)+(out,) -> (batch,out)
    return z2

X = np.random.randn(10, 5) # 10 samples, 5 features
W1 = np.random.randn(5, 8) * 0.1 # layer 1: 5 -> 8
b1 = np.zeros(8)
W2 = np.random.randn(8, 1) * 0.1 # layer 2: 8 -> 1
b2 = np.zeros(1)

y_pred = forward_pass(X, W1, b1, W2, b2)
print(y_pred.shape) # (10, 1)
```

The thing I always remind myself: without the ReLU in between, $(X @ W1) @ W2$ would mathematically collapse into a single matrix multiplication $X @ (W1 @ W2)$ — which is just linear regression again, no matter how many layers you stack. The non-linearity is what actually gives the network its extra expressive power.

Section D — Database Scenario-Based Questions (2 Coding)

Q9. [Database Scenario — Coding]

You have a users table stored as a pandas DataFrame, where each row has a user_id and an embedding column containing a 50-dimensional numpy array representing that user's preferences (learned from a recommender model). Write code to find the top-5 most similar users to a given target user, using cosine similarity.

Answer

This is a really common pattern in recommendation systems — you've already got embeddings sitting in a database/dataframe, and it's just a matter of stacking them into a matrix and reusing the same cosine similarity logic we always use. The main extra step compared to a plain numpy exercise is pulling the embeddings OUT of the DataFrame column into a clean 2D array first.

```
import numpy as np
import pandas as pd

# Simulated users table: user_id + 50-dim embedding stored as arrays
np.random.seed(0)
users_df = pd.DataFrame({
    'user_id': range(200),
    'embedding': [np.random.randn(50) for _ in range(200)]
})

def top_k_similar_users(users_df, target_user_id, k=5):
    # Stack all embeddings into one (n_users, 50) matrix
    embedding_matrix = np.vstack(users_df['embedding'].values) # (200, 50)
    user_ids = users_df['user_id'].values

    target_idx = np.where(user_ids == target_user_id)[0][0]
    target_vec = embedding_matrix[target_idx]

    # Cosine similarity of target against ALL users in one shot
    norms = np.linalg.norm(embedding_matrix, axis=1)
    target_norm = np.linalg.norm(target_vec)
    sims = (embedding_matrix @ target_vec) / (norms * target_norm)

    # Exclude the user themselves, then take the top-k highest similarity
    sims[target_idx] = -np.inf
    top_k_idx = np.argsort(sims)[::-1][:k]

    return pd.DataFrame({
        'user_id': user_ids[top_k_idx],
        'similarity': sims[top_k_idx].round(4)
    })

print(top_k_similar_users(users_df, target_user_id=42, k=5))
```

I deliberately compute similarity against ALL users at once with one matrix-vector multiplication (`embedding_matrix @ target_vec`) rather than looping row by row — that's the same broadcasting/matrix-multiplication mindset from the chapter, just applied on top of a database table instead of a raw numpy array. At real recommender-system scale (millions of users) you'd swap this for an approximate nearest-neighbour index like FAISS, but the underlying math — normalize, then dot product — is identical.

Q10. [Database Scenario — Coding]

You have an orders table (DataFrame) with columns [user_id, item_id, quantity, price]. Build a user-item interaction matrix and use matrix multiplication to compute total quantity-weighted revenue per user, then per item.

Answer

This is a nice example of how a 'boring' database table is secretly just a sparse matrix waiting to happen. Once I pivot the orders table into a (users x items) matrix, all the aggregate questions — total revenue per user, total revenue per item — become simple matrix-vector products instead of groupby loops.

```
import numpy as np
import pandas as pd

# Simulated orders table
orders_df = pd.DataFrame({
    'user_id': [1, 1, 2, 2, 3, 3, 1],
    'item_id': ['A', 'B', 'A', 'C', 'B', 'C', 'C'],
    'quantity': [2, 1, 3, 1, 2, 4, 1],
    'price': [10, 20, 10, 15, 20, 15, 15],
})

# Build a (users x items) QUANTITY matrix via pivot -- this IS the matrix from
# Section 1.2
qty_matrix = orders_df.pivot_table(
    index='user_id', columns='item_id', values='quantity', aggfunc='sum',
    fill_value=0
)
print(qty_matrix)
# A B C
# user_id
# 1 2 1 1
# 2 3 0 1
# 3 0 2 4

# Get one price per item (assume price is consistent per item here)
item_prices =
orders_df.drop_duplicates('item_id').set_index('item_id')['price']
item_prices = item_prices.reindex(qty_matrix.columns) # align column order!

# Revenue per user = quantity_matrix @ price_vector -- exactly Section
# 1.5/1.8's y = X @ w
revenue_per_user = qty_matrix.values @ item_prices.values
print(pd.Series(revenue_per_user, index=qty_matrix.index,
name='total_revenue'))
# user 1: 2*10 + 1*20 + 1*15 = 55
# user 2: 3*10 + 0*20 + 1*15 = 45
# user 3: 0*10 + 2*20 + 4*15 = 100
```

```
# Revenue per item = column sums weighted by price
revenue_per_item = qty_matrix.sum(axis=0).values * item_prices.values
print(pd.Series(revenue_per_item, index=qty_matrix.columns,
name='item_revenue'))
```

The part I think is most worth pointing out: `qty_matrix.values @ item_prices.values` is EXACTLY the $\hat{y} = X @ w$ pattern from the chapter — here X is the user-item quantity matrix and w is the price vector, and the matrix multiplication gives every user's total spend in one shot. The one gotcha I always double-check with pivot tables is column alignment — `item_prices` needs to be in the SAME column order as `qty_matrix.columns`, or you'll silently multiply quantities by the wrong item's price. That `reindex` call is doing the safety work there.

ANSWER KEY QUICK REFERENCE — MCQs

Level 1 (Warmup): Q1-C Q2-B Q3-A Q4-B Q5-B

Level 2 (Interviewer): Q1-B Q2-B Q3-B

All Descriptive, Coding, and Scenario-based answers are provided in full within their respective boxes above, written in the conversational interview-style format.